

Relatório Explicativo - Parte 4

Visão geral

Este documento explica, de forma simples e passo a passo, o que foi feito na Parte 4 do projeto, por que cada decisão foi tomada e como interpretar os resultados.

A Parte 4 teve como foco um problema de classificação supervisionada. O objetivo foi usar a base tratada do DATASUS para identificar indícios de câncer de mama.

Em outras palavras, queríamos responder à seguinte pergunta:

Dado um registro hospitalar da base tratada, o modelo consegue indicar se há ou não indício de câncer de mama?

1. De onde partimos

Antes da Parte 4, o projeto já havia passado por etapas importantes:

- conversão dos arquivos originais do DATASUS
- consolidação da base
- redução de colunas
- tratamento dos dados
- criação de variáveis derivadas

Ao final da Parte 2, chegamos ao arquivo `2025_tratado.parquet`, que foi a base usada na modelagem supervisionada.

Esse ponto é importante porque a Parte 4 não começou do zero. Ela aproveitou uma base já preparada, mais limpa e mais adequada para análise.

2. Qual era o objetivo da Parte 4

O objetivo foi transformar uma informação clínica já presente na base em um problema de classificação binária.

Usamos a variável `CANCER_MAMA_NIVEL` para criar o target:

- ``0`` = sem indício
- ``1`` = com indício

No código, isso ficou assim:

```
TARGET = (CANCER_MAMA_NIVEL > 0)
```

Esse passo foi importante porque permitiu transformar a análise em um problema clássico de Machine Learning supervisionado, exatamente como foi estudado nas aulas sobre classificação.

3. O primeiro achado importante: a base era muito desbalanceada

Logo no começo da análise, percebemos que a base tinha uma proporção muito pequena de casos positivos.

Em números aproximados:

- cerca de 99% dos registros eram da classe `0`
- cerca de 1% dos registros eram da classe `1`

Isso significa que o problema era fortemente desbalanceado.

Na prática, esse cenário é perigoso porque um modelo pode parecer "bom" só por prever quase tudo como negativo. Nesse caso, ele teria uma acurácia alta, mas não seria útil para detectar os casos que realmente importam.

4. Como tratamos esse desbalanceamento

Nós não ignoramos esse problema. Algumas decisões da modelagem foram feitas justamente para lidar com isso.

As principais medidas foram:

4.1. Uso de stratify no split

Na separação entre treino e teste, usamos:

```
train_test_split(..., stratify=y)
```

Isso garantiu que a proporção de positivos e negativos fosse preservada tanto no treino quanto no teste.

4.2. Uso de class_weight='balanced'

Na Regressão Logística, usamos:

```
class_weight='balanced'
```

Isso faz o algoritmo dar mais atenção à classe minoritária. Como os casos positivos eram raros, o modelo passou a penalizar mais os erros cometidos nessa classe.

4.3. Priorização de métricas corretas

Como a base era muito desbalanceada, decidimos não confiar apenas em accuracy.

As métricas mais importantes passaram a ser:

- `recall`
- `precision`
- `F1-score`
- matriz de confusão

4.4. Ajuste de threshold

Também não deixamos o modelo preso ao corte padrão de 0,5 quando ele retornava probabilidades.

Em vez disso, usamos um conjunto de validação para escolher um threshold mais adequado ao problema.

5. Como escolhemos as variáveis do modelo

Outro cuidado importante foi evitar vazamento de target.

Vazamento acontece quando usamos no modelo uma variável que, na prática, já carrega a resposta. Se isso acontece, o modelo parece ótimo, mas só porque teve acesso indireto à solução.

Por isso, evitamos colunas como:

- `CANCER\_MAMA\_NIVEL`
- `DIAG\_PRINC`
- `DIAGSEC\*`
- `CID\_MORTE`
- `DIAG\_PRINC\_GRUPO`

Em vez disso, priorizamos variáveis mais seguras, como:

- idade
- dias de permanência
- quantidade de diárias
- uso de UTI
- sexo
- raça/cor
- variáveis administrativas com baixa cardinalidade

Também removemos colunas com cardinalidade alta quando elas traziam custo computacional exagerado, como PROC_REA.

6. Por que usamos Pipeline e ColumnTransformer

Seguindo a abordagem estudada nas aulas, organizamos o pré-processamento com:

- `Pipeline`
- `ColumnTransformer`

Isso foi importante para manter o fluxo consistente entre treino e teste.

As colunas numéricas receberam:

- imputação pela mediana
- `StandardScaler`

As colunas categóricas receberam:

- imputação por valor mais frequente
- `OneHotEncoder`

Essa etapa foi especialmente importante para o KNN, já que modelos baseados em distância são sensíveis à escala das variáveis.

7. Quais modelos testamos

Foram comparados três modelos:

7.1. Logistic Regression

Foi utilizada como baseline principal da classificação supervisionada.

O motivo foi:

- é um modelo clássico
- aparece no conteúdo estudado
- funciona bem como ponto de referência
- permite trabalhar com probabilidades e threshold

7.2. KNN

Foi incluído porque aparece diretamente nas aulas de classificação e porque depende de feature scaling, o que também reforça a aplicação prática do conteúdo.

Também fizemos um ajuste simples do valor de k, testando alguns vizinhos possíveis.

7.3. Random Forest

Foi incluído porque os modelos baseados em árvores também fazem parte do material estudado.

O Random Forest entrou como alternativa para comparar uma abordagem diferente:

- menos linear que a regressão logística
- mais robusta para padrões complexos
- capaz de gerar probabilidades para análise via ROC/AUC

8. Como escolhemos os hiperparâmetros

8.1. KNN

No KNN, testamos alguns valores de k:

- 3
- 5
- 9
- 15

O objetivo foi ver qual valor entregava melhor comportamento na validação.

8.2. Threshold dos modelos probabilísticos

Para Logistic Regression e Random Forest, não escolhemos o threshold "no chute".

Usamos as probabilidades previstas no conjunto de validação e analisamos a curva precision-recall.

A partir dela, escolhemos o threshold com melhor comportamento segundo o critério definido.

Esse ponto é importante porque mostra que a decisão foi orientada por dados, e não arbitrária.

9. Como avaliamos os modelos

A avaliação foi feita em várias camadas.

9.1. Métricas principais

Usamos:

- `accuracy`
- `precision`
- `recall`
- `F1-score`
- matriz de confusão

9.2. Validação cruzada

Também incluímos validação cruzada para comparar os modelos com mais estabilidade.

Isso ajuda a reduzir a dependência de um único split de treino e teste.

9.3. ROC e AUC

Para os modelos probabilísticos, também avaliamos:

- curva ROC
- AUC

Isso ajudou a entender o poder de separação geral entre as classes.

10. O que aprendemos com os resultados

Os resultados mostraram que não existe um "melhor modelo" absoluto sem antes definir o critério de negócio.

Logistic Regression

A Regressão Logística se destacou principalmente em recall.

Isso significa que ela foi melhor para encontrar os casos positivos, ou seja, foi melhor para reduzir falsos negativos.

Esse comportamento é especialmente importante em saúde.

Random Forest

O Random Forest apresentou melhor equilíbrio geral em alguns cenários, além de melhor AUC.

Isso significa que ele teve boa capacidade de separação probabilística entre as classes.

KNN

O KNN teve desempenho bem inferior para o objetivo da tarefa.

Apesar de boa acurácia em alguns casos, ele encontrou poucos positivos reais.

11. Qual foi o racional no contexto de saúde

Esse foi o ponto central da interpretação.

No contexto deste problema, o custo dos erros não é simétrico.

Não é a mesma coisa:

- marcar alguém sem câncer como suspeito
- deixar passar alguém com indício real de câncer

Por isso, o grupo adotou a lógica correta:

é mais aceitável gerar falsos positivos do que falsos negativos

Em uma triagem inicial, isso faz sentido porque:

- o falso positivo pode ser encaminhado para uma avaliação mais aprofundada
- o falso negativo corre o risco de não receber atenção a tempo

Por esse motivo, o recall passou a ter um peso central na análise.

12. O que significa cada métrica na prática

Accuracy

Mostra o percentual total de acertos.

Problema:

em base desbalanceada, pode enganar.

Precision

Mostra quantos dos positivos previstos realmente eram positivos.

Quando a precision é baixa, o modelo está gerando muitos falsos positivos.

Recall

Mostra quantos positivos reais foram encontrados pelo modelo.

Quando o recall é alto, o modelo deixa passar poucos casos positivos.

F1-score

É uma métrica de equilíbrio entre precision e recall.

Ela ajuda quando queremos avaliar os dois ao mesmo tempo.

AUC

Resume a capacidade de separação do modelo em diferentes thresholds.

Quanto mais perto de 1, melhor.

13. Conclusão final

A Parte 4 mostrou que o trabalho não era apenas treinar classificadores, mas escolher uma estratégia coerente com o problema.

Os principais pontos da conclusão foram:

- a base era fortemente desbalanceada
- isso exigiu cuidado na escolha das métricas
- accuracy sozinha não era suficiente
- recall se tornou a métrica mais importante
- Logistic Regression e Random Forest foram os modelos mais relevantes
- KNN teve desempenho inferior para o objetivo de saúde
- a escolha final do melhor modelo depende do critério adotado:
- se o foco é detectar o máximo de positivos, a Regressão Logística tende a ser mais adequada
- se o foco é melhor separação probabilística geral, o Random Forest pode se destacar

14. Mensagem final para apresentação

Se for preciso resumir toda a Parte 4 em poucas frases:

Na Parte 4, transformamos a variável de indício de câncer de mama em um target binário e construímos uma etapa de classificação supervisionada. Selecionamos features sem vazamento de target, aplicamos pré-processamento com pipeline, testamos Regressão Logística, KNN e Random Forest, e avaliamos os modelos com métricas adequadas para bases desbalanceadas. Como o contexto é saúde, priorizamos recall, já que falsos negativos são mais graves do que falsos positivos. O principal racional da etapa foi ajustar a modelagem para maximizar a utilidade clínica da triagem, e não apenas a acurácia global.